

Visualization 模块说明

本文件夹包含所有可视化相关的模块，用于在 MuJoCo Viewer 中渲染 Ghost、外力箭头和奖励曲线。

📁 模块列表

文件	功能	MuJoCo 渲染方法
ghost.py	Ghost 渲染器	mjv_initGeom() + mjv_addGeoms()
force_visualizer.py	外力可视化器	mjv_initGeom() + mjv_connector()
force_applicator.py	外力施加器	data.xfrc_applied
reward_calculator.py	奖励计算器	N/A (纯计算)
reward_visualizer.py	奖励图表	MjvFigure + mjv_updateFigure()

🎨 MuJoCo 渲染方法详解

1. Ghost 渲染 (ghost.py)

原理：通过创建独立的 `MjModel` 和 `MjData`，设置半透明绿色材质，然后将几何体添加到场景中。

关键 API：

```
# 1. 创建 Ghost 模型 (深拷贝主模型)
ghost_model = copy.deepcopy(model)

# 2. 设置半透明绿色材质
ghost_color = np.array([0.5, 0.7, 0.5, 0.5], dtype=np.float32) # RGBA
ghost_model.geom_rgba[:] = ghost_color

# 3. 创建 Ghost 数据
ghost_data = mujoco.MjData(ghost_model)

# 4. 启用透明渲染
vopt = mujoco.MjvOption()
vopt.flags[mujoco.mjtVisFlag.mjVIS_TRANSPARENT] = True

# 5. 设置 Ghost 姿态
ghost_data.qpos[:] = reference_qpos # 参考运动的姿态

# 6. 前向运动学计算
mujoco.mj_forward(ghost_model, ghost_data)

# 7. 初始化几何体并添加到场景
for i in range(ghost_model.ngeom):
    if viewer_scene.ngeom >= viewer_scene.maxgeom:
        break
    geom = viewer_scene.geoms[viewer_scene.ngeom]
```

```

    mujoco.mjv_initGeom(
        geom,
        mujoco.mjtGeom.mjGEOM_NONE,
        np.zeros(3),
        np.zeros(3),
        np.zeros(9),
        ghost_color
    )

# 8. 添加 Ghost 几何体到场景
mujoco.mjv_addGeoms(
    ghost_model,
    ghost_data,
    vopt,
    pert,
    mujoco.mjtCatBit.mjCAT_DYNAMIC.value,
    viewer_scene
)

```

效果：半透明绿色的参考机器人姿态，叠加在当前机器人上方。

2. 外力可视化(`force_visualizer.py`)

原理：使用 `mjv_connector()` 创建连接两点的 CAPSULE（圆柱体），再用 SPHERE（球体）标记终点。

关键 API：

```

# 1. 获取施力点位置
body_pos = data.xpos[body_id] # body 的世界坐标

# 2. 计算箭头终点
force_magnitude = np.linalg.norm(force_vector)
force_direction = force_vector / force_magnitude
arrow_length = force_magnitude * force_scale # 力大小 → 箭头长度
force_end = body_pos + force_direction * arrow_length

# 3. 创建 CAPSULE 几何体（箭头主体）
if viewer_scene.ngeom < viewer_scene.maxgeom:
    capsule_geom = viewer_scene.geoms[viewer_scene.ngeom]
    mujoco.mjv_initGeom(
        capsule_geom,
        mujoco.mjtGeom.mjGEOM_CAPSULE,
        np.zeros(3),
        np.zeros(3),
        np.zeros(9),
        purple_color # RGBA = [0.8, 0.0, 0.8, 0.6]
    )

# 4. 使用 mjv_connector 连接两点
mujoco.mjv_connector(
    capsule_geom,

```

```

        mujoco.mjtGeom.mjGEOM_CAPSULE,
        capsule_radius, # 圆柱体半径
        body_pos[0], body_pos[1], body_pos[2], # 起点
        force_end[0], force_end[1], force_end[2] # 终点
    )

    viewer_scene.ngeom += 1

# 5. 创建 SPHERE 几何体 (箭头终点标记)
if viewer_scene.ngeom < viewer_scene.maxgeom:
    sphere_geom = viewer_scene.geoms[viewer_scene.ngeom]
    mujoco.mjv_initGeom(
        sphere_geom,
        mujoco.mjtGeom.mjGEOM_SPHERE,
        force_end, # 球心位置
        np.zeros(3), # 无旋转
        np.array([sphere_radius] * 3), # 球体大小
        purple_color
    )

    viewer_scene.ngeom += 1

```

效果：紫色箭头从 body 指向力的终点，箭头长度与力大小成正比。

特殊模式：

- **恒定力模式：** `force_scale = 0.02` (1N = 2cm 长度)
- **弹簧力模式：** `force_scale = 1/k` (箭头终点 = 引力中心位置)

3. 外力施加 (`force_applicator.py`)

原理：直接修改 `data.xfrc_applied` 数组，在指定 body 上施加外力。

关键 API：

```

# 1. 施加外力 (恒定力模式)
body_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, body_name)
data.xfrc_applied[body_id, :3] = force_vector # [fx, fy, fz]
data.xfrc_applied[body_id, 3:] = 0.0 # 无力矩

# 2. 施加弹簧力 (Spring 模式)
body_pos = data.xpos[body_id]
displacement = reference_pos - body_pos # 位移向量
spring_force = k * displacement # 胡克定律 F = k * Δx
data.xfrc_applied[body_id, :3] = spring_force

```

两种模式：

模式	力的计算	应用场景
----	------	------

模式	力的计算	应用场景
constant	恒定力向量	推力、风力等
spring	弹簧力 $F = k * (target - pos)$	牵引、拉回等

三种停止方式：

停止方式	行为	应用场景
fixtime	固定时间后自动停止	短暂干扰
keeping	持续施加，按键切换	长时间干扰
None	立即停止	测试

4. 奖励可视化 (reward_visualizer.py)

原理：使用 MuJoCo 原生的 MjvFigure API 在 Viewer 侧边显示曲线图。

关键 API：

```
# 1. 创建 Figure 对象
fig = mujoco.MjvFigure()
mujoco.mjv_defaultFigure(fig)

# 2. 设置图表属性
fig.title = "pos_tracking_global" # 图表标题

# 3. 设置坐标轴范围 (2x2 数组)
fig.range[0][0] = -300 # x 轴最小值
fig.range[0][1] = 0    # x 轴最大值
fig.range[1][0] = -0.01 # y 轴最小值
fig.range[1][1] = 0.01 # y 轴最大值

# 4. 设置颜色
fig.figurergba[:] = [1.0, 1.0, 1.0, 0.3] # 白色半透明背景
fig.panergba[:] = [1.0, 1.0, 1.0, 0.5]   # 白色半透明面板
fig.linergb[0][:] = [0.0, 1.0, 0.0]      # 绿色线条
fig.gridrgb[:] = [0.7, 0.7, 0.7]          # 灰色网格
fig.textrgb[:] = [0.0, 0.0, 0.0]         # 黑色文字

# 5. 更新数据
fig.linepnt[0] = len(history) # 数据点数量
fig.linedata[0][0:2*len(history)] = xy_data.flatten() # [x0,y0,x1,y1,...]

# 6. 更新 Figure (自动调整范围)
mujoco.mjv_updateFigure(
    fig,
    mujoco.mjtGridPos.mjGRID_BOTTOMLEFT, # 位置 (左下角)
    rect, # 图表矩形区域 [left, bottom, width, height]
```

```
viewer_scene
)
```

数据结构：

- `fig.linedata[line_idx]`: 长度为 2000 的一维数组，存储 `[x0, y0, x1, y1, ...]`
- `fig.linepnt[line_idx]`: 该线的数据点数量（最多 1000 点）
- 多条线：`line_idx ∈ [0, 4]`，最多 5 条线

预设颜色：

奖励项	颜色	RGB
<code>smoothness</code>	红色	<code>[1.0, 0.0, 0.0]</code>
<code>pos_tracking_global</code>	绿色	<code>[0.0, 1.0, 0.0]</code>
<code>pos_tracking_local</code>	蓝色	<code>[0.0, 0.0, 1.0]</code>
<code>quat_tracking_global</code>	黄色	<code>[1.0, 1.0, 0.0]</code>
<code>quat_tracking_local</code>	品红	<code>[1.0, 0.0, 1.0]</code>
其他	浅蓝色	<code>[0.5, 0.5, 1.0]</code>

 自定义渲染

添加新的几何体类型

MuJoCo 支持的几何体类型（`mujoco.mjtGeom`）：

类型	枚举值	用途
<code>mjGEOM_PLANE</code>	0	平面
<code>mjGEOM_HFIELD</code>	1	高度场
<code>mjGEOM_SPHERE</code>	2	球体
<code>mjGEOM_CAPSULE</code>	3	胶囊体（圆柱+半球）
<code>mjGEOM_ELLIPSOID</code>	4	椭球体
<code>mjGEOM_CYLINDER</code>	5	圆柱体
<code>mjGEOM_BOX</code>	6	立方体
<code>mjGEOM_MESH</code>	7	自定义网格

示例：渲染自定义球体

```
if viewer_scene.ngeom < viewer_scene.maxgeom:
    geom = viewer_scene.geoms[viewer_scene.ngeom]
```

```
# 初始化几何体
mujoco.mjv_initGeom(
    geom,
    mujoco.mjtGeom.mjGEOM_SPHERE,      # 球体
    np.array([0.0, 0.0, 1.0]),          # 位置 [x, y, z]
    np.zeros(3),                        # 旋转 (四元数或轴角)
    np.array([0.1, 0.1, 0.1]),          # 大小 [rx, ry, rz]
    np.array([1.0, 0.0, 0.0, 0.8])      # 颜色 RGBA
)

viewer_scene.ngeom += 1
```

示例：渲染自定义箭头

```
# 使用 mjv_connector 创建两点之间的连接
mujoco.mjv_connector(
    geom,
    mujoco.mjtGeom.mjGEOM_CAPSULE,      # 或 mjGEOM_CYLINDER
    width,                               # 连接宽度
    from_x, from_y, from_z,              # 起点
    to_x, to_y, to_z                    # 终点
)
```

参考资料

- **MuJoCo 官方文档**: <https://mujoco.readthedocs.io/en/stable/APIreference/>
 - `mjv_initGeom`: 初始化几何体
 - `mjv_addGeoms`: 添加模型几何体到场景
 - `mjv_connector`: 创建连接几何体
 - `mjv_updateFigure`: 更新图表
 - `MjvFigure`: 图表数据结构
 - `MjvScene`: 场景数据结构
- **原始参考项目**:
 - `deploy_robot-main-v1.0.0/deploy_robot/sim/viewer_plus/`
 - Ghost 渲染: `viewer_plus.py`
 - 奖励图表: `reward_plotter.py`

设计要点

1. 场景容量限制:

- `viewer_scene.maxgeom`: 最大几何体数量 (通常 10000)
- 渲染前检查: `if viewer_scene.ngeom < viewer_scene.maxgeom`

- 渲染后递增: `viewer_scene.ngeom += 1`

2. 颜色透明度:

- 使用 RGBA 格式: `[R, G, B, A]`, 值域 `[0.0, 1.0]`
- 透明度 `A = 0.5` 表示半透明
- 启用透明渲染: `vopt.flags[mujoco.mjtVisFlag.mjVIS_TRANSPARENT] = True`

3. 坐标系:

- 所有位置均为**世界坐标系**
- `data.xpos[body_id]`: body 的世界坐标位置
- `data.xfric_applied[body_id]`: 施加在 body 上的外力 (世界坐标系)

4. 性能优化:

- 每帧重新初始化几何体 (不累积)
- Ghost: 约 30-50 个几何体/环境
- 外力箭头: 2 个几何体/环境
- 奖励图表: 不占用几何体容量 (独立 API)

使用示例

完整渲染流程

```
# 初始化
ghost_renderer = GhostRenderer(model)
force_visualizer = ForceVisualizer()
reward_plotter = RewardPlotter(history_length=300)

# 注册奖励项
reward_plotter.register_terms(['pos_tracking', 'quat_tracking'])

# 主循环
with mujoco.viewer.launch_passive(...) as viewer:
    while viewer.is_running():
        # 1. 设置 Ghost 姿态
        ghost_renderer.set_ghost_qpos(reference_qpos)

        # 2. 渲染 Ghost
        ghost_renderer.render_ghost(viewer.user_scn)

        # 3. 渲染外力箭头
        force_visualizer.render_force_arrow(
            viewer.user_scn,
            model,
            data,
            body_id,
            force_vector
        )
```

```
# 4. 更新奖励图表
rewards = compute_rewards(data, motion_data)
reward_plotter.update(rewards)

# 5. 同步到 Viewer
viewer.sync()
```

注意事项

1. **Ghost 渲染必须在每帧调用**：`mjv_addGeoms()` 不会持久化几何体
2. **外力箭头会自动清除**：每帧 `viewer.user_scn` 会重置 `ngeom`
3. **奖励图表持久化**：调用 `mjv_updateFigure()` 后会保持显示
4. **多环境渲染**：每个环境独立调用，使用不同的 `body_id` 和 `data`

最后更新: 2025-11-12